

Оглавление

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	6
1 Аналитический раздел	8
1.1 Анализ предметной области	8
1.2 Сравнительный анализ существующих систем	8
1.3 Требования к проектируемой системе	9
1.3.1 Функциональные требования	9
1.3.2 Нефункциональные требования	10
1.3.3 Ограничения	10
1.4 Метрики эффективности и правила их расчёта	10
1.5 Сущности предметной области, роли пользователей и варианты использования .	12
1.5.1 Сущности и связи	12
1.5.2 Роли пользователей и варианты использования	13
1.6 Выбор модели данных и архитектуры хранилища	14
1.6.1 Выбор логической модели данных	14
1.6.2 Выбор физической архитектуры (OLTP vs OLAP)	15
2 Конструкторский раздел	17
2.1 Логическое проектирование базы данных	17
2.1.1 Соответствие нормальным формам	21
2.1.2 Проектирование представлений	21
2.1.3 Проектирование индексов	22
2.2 Проектирование ограничений целостности	22
2.3 Проектирование алгоритмов обработки данных	23
2.3.1 Алгоритмы расчёта метрик игроков	23
2.3.2 Алгоритм агрегирования командной статистики	23
2.3.3 Алгоритм массового обновления агрегатов	24
2.4 Проектирование ролевой модели доступа	24
2.5 Проектирование программного обеспечения и кэширования	25
2.5.1 Архитектура приложения	25
2.5.2 Спецификация REST API	25
2.5.3 Кэширование результатов запросов	27

3	Технологический раздел	29
3.1	Выбор и обоснование средств реализации	29
3.1.1	СУБД	29
3.1.2	Кэширование	29
3.1.3	Серверный слой	29
3.1.4	Клиентский слой	30
3.2	Физическая реализация базы данных	30
3.2.1	Создание таблиц и ограничений	30
3.2.2	Реализация индексов	30
3.2.3	Загрузка тестовых данных	31
3.3	Реализация логики предметной области в СУБД	31
3.3.1	Хранимые функции расчёта метрик	31
3.3.2	Триггеры логической целостности	32
3.4	Реализация ролевой модели доступа	33
3.5	Реализация программного обеспечения и интерфейса	33
3.5.1	Серверное приложение	33
3.5.2	Кэширование	34
3.5.3	Пользовательский интерфейс	34
3.6	Тестирование и верификация аналитических данных	37
4	Исследовательский раздел	39
4.1	Цель и методика исследования	39
4.2	Влияние объёма данных и стратегий индексации	39
4.2.1	Влияние объёма данных на время выполнения запроса	39
4.2.2	Исследование стратегий индексации	39
4.3	Сравнение витрины и прямой агрегации по таблице фактов	40
4.4	Исследование эффективности кэширования	41
4.5	Исследование производительности при параллельной нагрузке	42
4.6	Анализ результатов и рекомендации	42
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	44
	ПРИЛОЖЕНИЯ	46
	Приложение А. Скрипты создания структуры базы данных	46
	Приложение Б. Скрипты создания функций и процедур	46
	Приложение В. Исходный код механизма кэширования	47

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

Спортивные метрики и обозначения

AST	передачи (assists)
BLK	блок-шоты (blocks)
BPM	вклад игрока (Box Plus/Minus)
eFG%	эффективный процент с игры (effective field goal percentage)
FG3A	попытки трёхочкового броска
FG3M	попадания трёхочковых бросков
FGA	попытки броска с игры (field goals attempted)
FGM	попадания с игры (field goals made)
FTA	попытки штрафного броска (free throws attempted)
FTM	попадания штрафных бросков (free throws made)
MP	минуты на площадке (minutes played)
NBA	Национальная баскетбольная ассоциация
PER	рейтинг эффективности игрока (Player Efficiency Rating)
PF	персональные фолы (personal fouls)
PTS	очки (points)
REB_DEF	подборы в защите
REB_OFF	подборы в нападении
STL	перехваты (steals)
TmFGA	командные попытки броска с игры
TmFTA	командные попытки штрафного броска
TmMP	суммарные минуты команды
TmTOV	командные потери
TOV	потери (turnovers)
TRB	подборы суммарные (total rebounds)
TS%	истинный процент бросков (true shooting percentage)
USG%	доля владений в атаке (usage percentage)

Технические

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
B-tree	сбалансированное дерево (balanced tree)
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DDL	Data Definition Language
ER	Entity-Relationship
ETL	Extract, Transform, Load
FK	внешний ключ (Foreign Key)
HTAP	Hybrid Transactional/Analytical Processing
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KPI	ключевые показатели эффективности (Key Performance Indicators)
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
P95	95-й перцентиль
PK	первичный ключ (Primary Key)
PL/pgSQL	процедурный язык PostgreSQL
REST	Representational State Transfer
SPA	одностраничное приложение (Single Page Application)
SQL	Structured Query Language
СУБД	система управления базами данных
UML	Unified Modeling Language
URL	Uniform Resource Locator
UPSERT	операция вставки с обновлением (UPDATE + INSERT)
1НФ / 2НФ / 3НФ	первая / вторая / третья нормальная форма

1 Аналитический раздел

1.1 Анализ предметной области

Национальная баскетбольная ассоциация (NBA) является источником одного из наиболее детализированных массивов спортивной статистики среди крупнейших мировых профессиональных лиг [1]. Для каждого матча фиксируются итоговый счёт, составы команд и числовые показатели по каждому игроку: очки, подборы, передачи, блок-шоты, перехваты и иные. На этой основе можно строить рейтинги, сопоставлять сезоны и выявлять тенденции развития игры.

В современной баскетбольной аналитике для комплексной оценки результативности применяются продвинутое метрики, стандартизированные в работах ведущих спортивных статистиков [2]. Расчёт таких показателей как истинный процент бросков TS%, эффективный процент с игры eFG%, доля владений в атаке USG%, рейтинг эффективности игрока PER и показатель вклада игрока BPM требует хранения непротиворечивых первичных данных по матчам, их агрегирования по сезонам и применения фиксированных правил вычисления.

Обработка спортивной статистики требует строгого контроля над целостностью данных: сумма очков игроков должна совпадать с командным счётом, агрегаты по сезонам должны вычисляться из одних и тех же первичных фактов, а параллельный доступ не должен нарушать консистентность записей. Это определяет необходимость разработки специализированного хранилища данных с поддержкой транзакционной целостности и декларативной валидации бизнес-правил.

1.2 Сравнительный анализ существующих систем

Перед проектированием собственного решения проведём анализ функциональных возможностей наиболее распространённых публичных систем сбора и анализа статистики NBA: официального портала NBA Stats [1], справочной платформы Basketball-Reference [3] и аналитического раздела ESPN [4].

В ходе анализа указанных систем критерии сравнения определены исходя из двух групп требований: необходимость воспроизводимого доступа к первичным данным и прозрачного расчёта метрик, обоснованная в [2], а также потребность в ролевом разграничении доступа как принципе проектирования многопользовательских информационных систем [5, с. 150]. Именно эти функции были выделены в качестве критериев для дальнейшего сравнения. В таблице 1.1 представлен сравнительный анализ существующих систем.

Таблица 1.1 — Сравнительный анализ существующих систем по функциональным возможностям

Критерий	NBA.com Stats	Basketball-Reference	ESPN Stats
Просмотр статистики матчей	+	+	+
Фильтрация по сезону, позиции, команде	частично	+	частично
Расчёт TS%, eFG%, PER, BPM	+(формулы скрыты)	+(формулы открыты [6, 7])	частично
Доступ к сырым данным через API (произвольные запросы)	–	–	–
Интерактивное сравнение стат. профилей игроков	частично	–	+
Ролевое разграничение доступа	–	–	–
Локальное хранение, воспроизводимость расчётов	–	–	–

Анализ показывает, что NBA.com Stats и ESPN предоставляют данные через готовый пользовательский интерфейс с непрозрачными формулами расчёта метрик. Basketball-Reference публикует описание своих формул [6, 7], однако не предоставляет открытого API для произвольных аналитических запросов. Ни одна из систем не поддерживает ролевого разграничения доступа и воспроизводимого локального хранения данных. Выявленные ограничения определяют требования к проектируемой системе, сформулированные в следующем разделе.

1.3 Требования к проектируемой системе

1.3.1 Функциональные требования

- 1) хранение справочных данных: игровых позиций, сезонов, клубов NBA;
- 2) хранение профилей игроков: персональные и антропометрические данные, позиция, сведения о драфте;
- 3) хранение данных о матчах: сезон, команды-участники, дата, итоговый счёт, статус;
- 4) хранение детализированной статистики игрока в матче: очки, подборы, передачи, броски с игры и штрафные, потери, фолы, время на площадке;
- 5) хранение агрегированной статистики игрока за сезон: средние значения, процентные показатели, расчётные метрики;
- 6) хранение истории выступлений игрока за клубы по сезонам;
- 7) расчёт командной статистики в рамках матча и сезона на основе индивидуальных показателей игроков;
- 8) автоматизированный расчёт метрик эффективности (eFG%, TS%, USG%, PER, BPM) на основе первичных фактов матчей;
- 9) предоставление сводных рейтингов игроков и командной статистики.

1.3.2 Нефункциональные требования

- 1) производительность: время отклика на аналитические запросы чтения (агрегация за сезон) не должно превышать 200 мс — рекомендуемый порог времени отклика сервера согласно спецификации Google PageSpeed Insights [8];
- 2) объём данных: система должна выдерживать нагрузку не менее 100 000 записей в таблице фактов (соответствует статистике за 5 регулярных сезонов);
- 3) целостность: база данных должна соответствовать требованиям ACID, обеспечивая ссылочную и логическую целостность на уровне СУБД;
- 4) безопасность: ролевое разграничение доступа (администратор, аналитик, читатель), реализованное средствами СУБД на уровне таблиц и представлений;
- 5) масштабируемость чтения: время отклика на повторяющиеся аналитические запросы при конкурентном доступе не должно превышать 200 мс при нагрузке до 50 одновременных подключений, что соответствует оценке совокупной аудитории системы: аналитический отдел, тренерский штаб и скауты клуба NBA.

1.3.3 Ограничения

- 1) глубина хранения данных ограничена 5 регулярными сезонами (с 2019/20 по 2023/24);
- 2) пользовательский интерфейс предназначен для просмотра и анализа данных, а не для ввода первичной статистики в режиме реального времени.

1.4 Метрики эффективности и правила их расчёта

Обозначения приведены в перечне сокращений.

Командная статистика не выделяется в отдельную сущность: она является производной от индивидуальных показателей и вычисляется динамически суммированием по всем игрокам команды в конкретном матче.

Эффективный процент с игры, eFG%. Метрика корректирует ценность трёхочкового броска. Согласно методологии NBA [9], формула имеет вид:

$$eFG\% = \frac{FGM + 0,5 \cdot FG3M}{FGA} \quad (1.1)$$

Истинный процент бросков, TS%. Показатель объединяет броски с игры и штрафные броски в единую метрику эффективности. Множитель 0,44 в знаменателе является эмпирически выведенным коэффициентом Дина Оливера, учитывающим долю штрафных бросков, завершающих владение [10]:

$$TS\% = \frac{PTS}{2 \cdot (FGA + 0,44 \cdot FTA)} \quad (1.2)$$

Доля владений в атаке, USG%. Оценивает процент командных владений, которые завершает конкретный игрок, находясь на площадке [2]. В базе данных показатель хранится как доля $[0; 1]$; в интерфейсе отображается в процентах ($\times 100$). Формула:

$$USG = \frac{(FGA + 0,44 \cdot FTA + TOV) \cdot (TmMP/5)}{MP \cdot (TmFGA + 0,44 \cdot TmFTA + TmTOV)} \quad (1.3)$$

Деление $TmMP$ на 5 нормирует командное время с учётом того, что на площадке одновременно находятся пять игроков.

Рейтинг эффективности игрока, PER. Оригинальная формула разработана Джоном Холлинджером [11]. В настоящей работе применяется её упрощённая версия без позиционных поправок, методология которой описана на портале Basketball-Reference [6]. Подборы $TRB = REB_OFF + REB_DEF$. Ненормализованный вклад приводится к шкале «за 48 минут», затем масштабируется к среднему по лиге, равному 15:

$$uPER_i = \left[PTS + TRB + AST + STL + BLK - TOV - PF - (FGA - FGM) - 0,44 \cdot (FTA - FTM) \right] \cdot \frac{48}{MP_i} \quad (1.4)$$

$$lg_aPER = AVG(uPER_i) \quad \text{по игрокам с } MP_i > 100 \quad (1.5)$$

$$PER_i = uPER_i \cdot \frac{15}{lg_aPER} \quad (1.6)$$

Минимальный порог $MP_i \geq 50$ минут за сезон; для расчёта lg_aPER учитываются игроки с $MP_i > 100$ минут.

Вклад игрока (Box Plus/Minus, BPM). Метрика разработана Дэниелом Майерсом [7]. В системе используется классическая линейная аппроксимация первой версии алгоритма (BPM 1.0) с нормировкой показателей на 100 минут ($X_{100} = X/MP \cdot 100$) и учётом перехватов, блоков и фолов. Итоговая формула:

$$\begin{aligned} raw_i = & 0,123 \cdot PTS_{100} + 0,234 \cdot TRB_{100} + 0,689 \cdot AST_{100} + 0,445 \cdot STL_{100} \\ & + 0,407 \cdot BLK_{100} - 0,605 \cdot TOV_{100} - 0,086 \cdot PF_{100} \end{aligned} \quad (1.7)$$

$$BPM_i = raw_i - AVG(raw_i) \quad \text{по игрокам с } MP_i > 100 \quad (1.8)$$

Среднее BPM по лиге после вычитания среднего равно 0. Минимальный порог для расчёта показателя игрока — $MP_i \geq 50$ минут за сезон.

1.5 Сущности предметной области, роли пользователей и варианты использования

1.5.1 Сущности и связи

На основе анализа предметной области и состава метрик раздела 1.4 выделены восемь основных сущностей.

- 1) позиция — справочник игровых позиций в номенклатуре NBA;
- 2) сезон — игровой год, его календарные границы и тип турнира;
- 3) команда — клуб NBA: конференция, арена;
- 4) игрок — персональные и антропометрические данные, позиция, сведения о драфте;
- 5) матч — сезон, команды-участники, дата, счёт, статус;
- 6) статистика игрока в матче — числовые показатели конкретного игрока в конкретной игре; основная таблица фактов предметной области;
- 7) статистика игрока за сезон — агрегаты и расчётные метрики по тройке «игрок – сезон – команда»;
- 8) история выступлений — связь «игрок – команда – сезон» с датами и типом контракта.

Связь «многие-ко-многим» между игроками и матчами реализуется через сущность «Статистика игрока в матче»: один игрок участвует во многих матчах, в одном матче фиксируется статистика многих игроков [5, 12].

Связь матча с сезоном и командами на ER-диаграмме представлена тремя бинарными связями: «Проходит в» связывает Сезон (1) и Матч (М); «Дом. команда» и «Гост. команда» связывают Команду (1) с Матчем (М) для хозяев и гостей соответственно [12]. Такое разбиение на бинарные связи соответствует реляционной схеме, где таблица games хранит season_id, home_team_id и away_team_id как три независимых внешних ключа.

Концептуальная ER-диаграмма в нотации Чена приведена на рисунке 1.1; на ней отражены сущности предметной области и бинарные связи между ними.

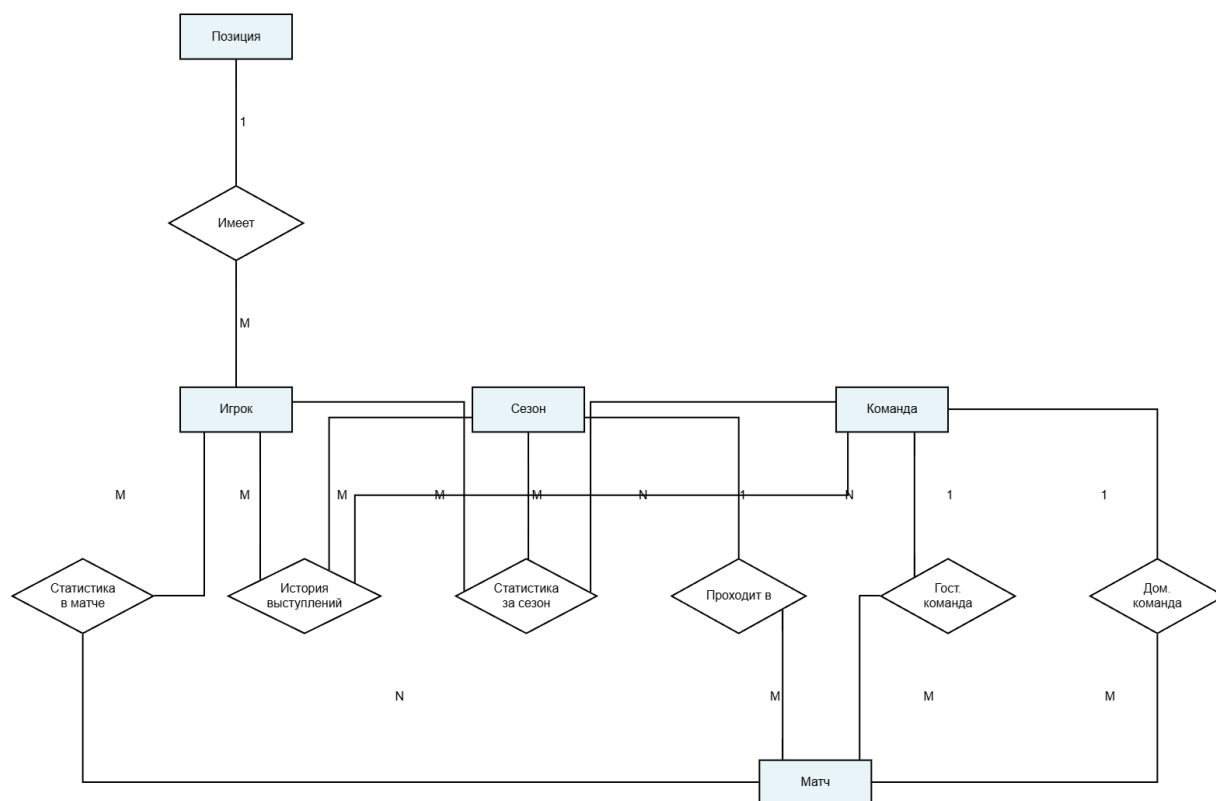


Рисунок 1.1 — Концептуальная ER-диаграмма

1.5.2 Роли пользователей и варианты использования

Система предусматривает три роли пользователей.

- 1) администратор — полное управление структурой и содержимым хранилища: создание и изменение схемы, загрузка и сопровождение данных, резервное копирование;
- 2) аналитик — чтение данных и запуск расчётных процедур для получения производных показателей;
- 3) читатель — доступ к агрегированным данным через представления; изменение содержимого базовых таблиц недоступно.

Соответствие ролей и допустимых операций задаётся тремя вариантами использования.

- 1) просмотр статистики — получение сведений о матчах, игроках и командах, агрегированных показателях и рейтингах; доступно всем ролям;
- 2) расчёт метрик — вычисление продвинутых показателей на основе первичных данных; доступно аналитику и администратору;
- 3) управление данными — загрузка данных о матчах, обновление справочников, изменение схемы; доступно только администратору.

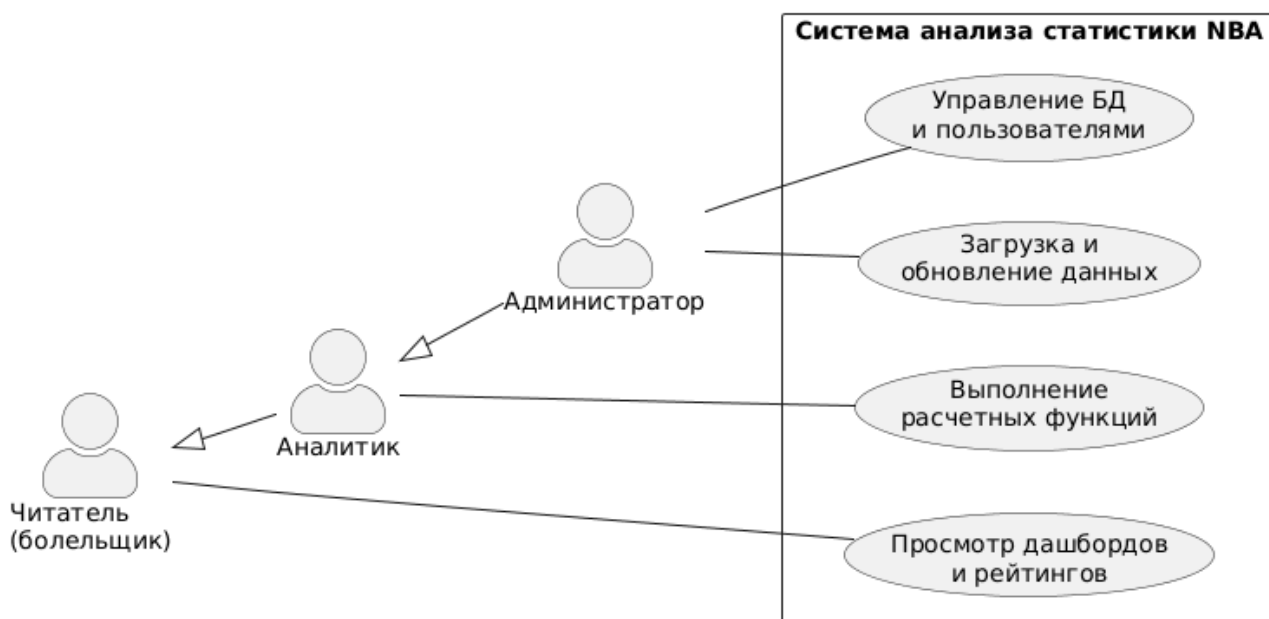


Рисунок 1.2 — Диаграмма вариантов использования

1.6 Выбор модели данных и архитектуры хранилища

Для выбора оптимального решения разделим задачу на два этапа: выбор логической модели данных и выбор физической архитектуры (профиля нагрузки) [13, с. 42].

1.6.1 Выбор логической модели данных

Рассмотрим основные логические модели данных для предметной области спортивной статистики.

Графовая модель данных (Neo4j). В данной модели данные представлены в виде узлов и рёбер (связей). Она оптимальна для обхода сложных социальных связей. Однако для баскетбольной статистики, где доминируют математические агрегации (вычисление средних, сумм, процентов) по сотням тысяч фактов, графовые СУБД демонстрируют крайне низкую производительность [13, с. 63–64].

Документно-ориентированная модель (MongoDB). Хранит данные в виде независимых иерархических структур (например, JSON). Как отмечают П. Садаладж и М. Фаулер [14, с. 35], такие базы данных оптимальны для агрегированных объектов без жёсткой схемы (schema-less). Сырая статистика матча (Box Score) естественно ложится в документ, однако агрегация данных по нескольким сезонам и связывание игроков с историей команд потребует сложных программных JOIN-операций, которые в документной модели неэффективны.

Реляционная модель данных. Представляет данные в виде двумерных таблиц с жёстко заданной схемой [5, с. 150]. Спортивная статистика обладает строгой, редко меняющейся структурой (игрок всегда имеет очки, подборы, фолы). Реляционная алгебра обеспечивает мощный математический аппарат для многомерных агрегаций, а сама модель (schema-on-write) предотвращает попадание аномальных данных (например, отрицательных очков) за счёт ссылочной

целостности и табличных ограничений.

Таблица 1.2 — Сравнительная характеристика логических моделей данных

Критерий	Графовая	Документная	Реляционная
Управление схемой	Отсутствует / Гибкая	Гибкая (schema-on-read)	Строгая (schema-on-write)
Математические агрегации фактов	Низкая эффективность	Средняя (через пайплайны)	Высокая (нативная оптимизация)
Связывание сущностей (JOIN)	Обход по ссылкам (быстро)	Затруднено (денормализация)	Нативная поддержка (внешние ключи)
Целостность фактов	На уровне приложения	Поддержка JSON Schema	Строгие констрейнты таблиц

На основе анализа логических моделей выбрана **реляционная модель**, так как данные строго структурированы и требуют сложных агрегаций с объединением справочников.

1.6.2 Выбор физической архитектуры (OLTP vs OLAP)

После выбора реляционной модели необходимо определить физический способ хранения данных: по строкам (Row-oriented) или по столбцам (Column-oriented).

Колоночные СУБД (ClickHouse). В данной архитектуре данные физически хранятся по колонкам, что многократно ускоряет агрегирующие аналитические запросы над миллионами записей (OLAP-профиль) [15]. Однако колоночные системы плохо справляются с точечными транзакционными обновлениями данных (UPDATE/DELETE).

Строковые транзакционные СУБД (PostgreSQL). Ориентированы на смешанную нагрузку (HTAP). Движок хранения гарантирует строгую согласованность данных (ACID-транзакции) на уровне физических страниц [16, с. 510].

Рассматриваемая система предполагает смешанный профиль: процедура обновления сезонной статистики выполняет ресурсоёмкие транзакционные операции слияния (UPSERT) для сотен записей. Также требуется ретроспективная корректировка протоколов матчей (точечные UPDATE). При целевом объёме таблицы фактов порядка 200 000 строк (5 сезонов) классическая строковая реляционная СУБД способна выполнять аналитические запросы без существенных задержек с помощью B-tree индексов. Отставание в скорости чтения сводных таблиц будет полностью нивелировано in-memoу кэшированием в Redis [17]. Использование колоночной СУБД при таком масштабе является архитектурной избыточностью.

Таким образом, для реализации хранилища целесообразно использовать реляционную СУБД со строковой физической архитектурой, дополненную in-memoу кэшированием.

Вывод

Сформулированы функциональные и нефункциональные требования и ограничения; определён перечень метрик и правила их вычисления. Выделены восемь сущностей, установлены связи «многие-ко-многим» через таблицы фактов, описаны три пользовательские роли.

По результатам сравнения логических и физических моделей данных для проектируемой системы выбрана реляционная модель со строковым хранением. В отличие от документоориентированных и графовых моделей, она позволяет эффективно выполнять многомерные агрегации фактов, а в отличие от колоночных систем — обеспечивает полную транзакционную целостность (ACID) при точечных и множественных обновлениях связанных таблиц.

Сравнительный анализ существующих систем (NBA.com Stats, Basketball-Reference, ESPN Stats) подтвердил отсутствие у них произвольных аналитических запросов, управляемого ролевого доступа и воспроизводимого локального хранения данных, что обуславливает актуальность и целесообразность разработки специализированной базы данных. ER-диаграмма и диаграмма вариантов использования приведены на рисунках 1.1 и 1.2.

2 Конструкторский раздел

2.1 Логическое проектирование базы данных

Инфологическая модель реализована реляционной схемой из восьми таблиц: факты — в `game_player_stats`, сезонные агрегаты — в `player_season_stats` (денормализованная витрина для ускорения чтения). В `seasons` ключ `season_id` и метка `label` («ГТТГ-ГТ»). Связи таблиц показаны на рисунке 2.1.

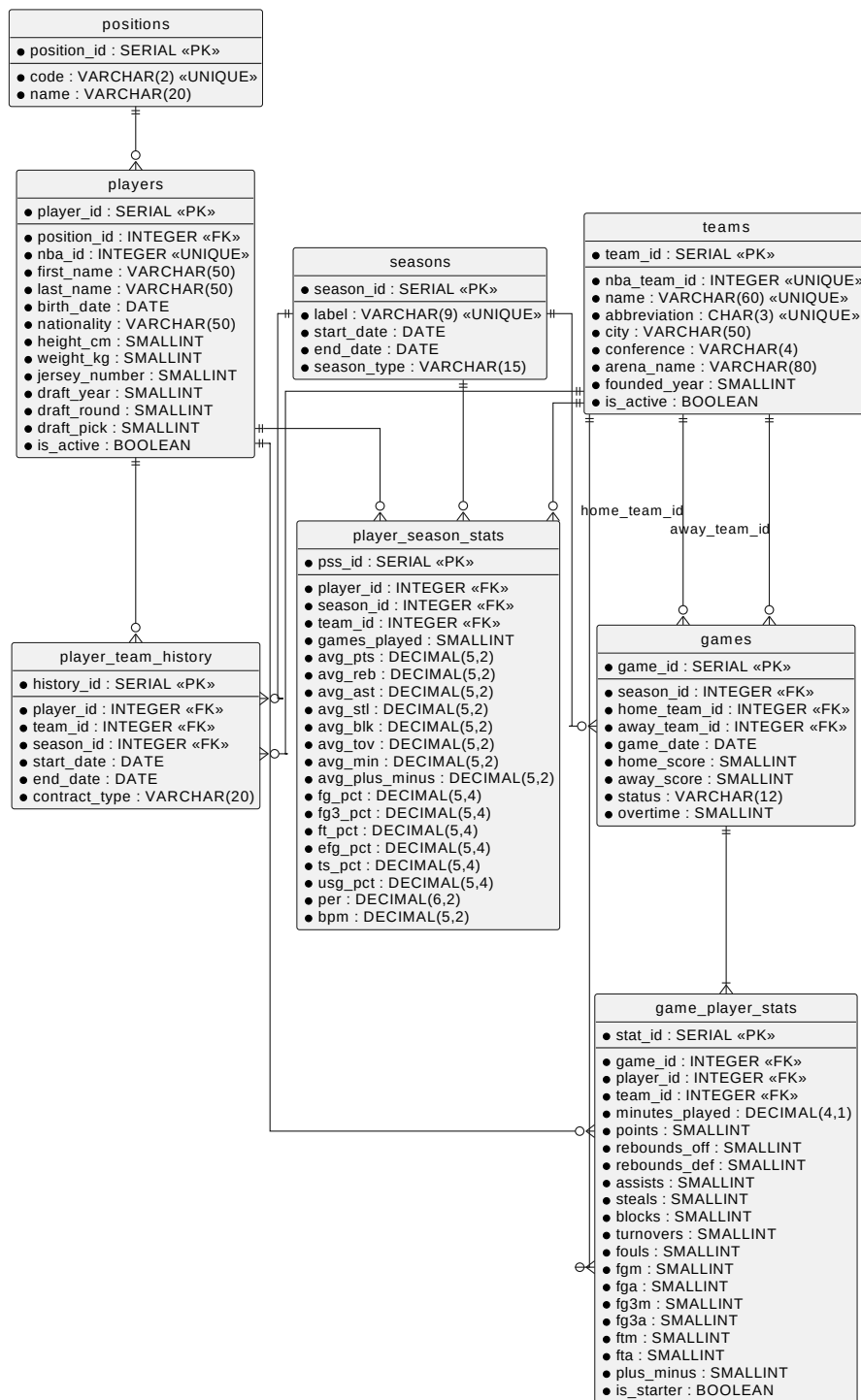


Рисунок 2.1 — Реляционная схема базы данных

Таблица 2.1 — Таблица positions (справочник позиций)

Атрибут	Тип	Ключ / назначение
position_id	SERIAL	PK
code	VARCHAR(2)	UNIQUE; код PG, SG, SF, PF, C
name	VARCHAR(20)	полное название позиции

Таблица 2.2 — Таблица seasons (сезоны)

Атрибут	Тип	Ключ / назначение
season_id	SERIAL	PK
label	VARCHAR(9)	UNIQUE; метка сезона (напр. 2023-24)
start_date	DATE	дата начала сезона
end_date	DATE	дата окончания сезона
season_type	VARCHAR(15)	Regular, Playoff, Preseason

Таблица 2.3 — Таблица teams (команды)

Атрибут	Тип	Ключ / назначение
team_id	SERIAL	PK
nba_team_id	INTEGER	UNIQUE; id на NBA.com
name	VARCHAR(60)	UNIQUE; полное название
abbreviation	CHAR(3)	UNIQUE; аббревиатура
city	VARCHAR(50)	город базирования
conference	VARCHAR(4)	CHECK IN ('East', 'West')
arena_name	VARCHAR(80)	домашняя арена
founded_year	SMALLINT	год основания
is_active	BOOLEAN	признак активной команды

Таблица 2.4 — Таблица players (игроки)

Атрибут	Тип	Ключ / назначение
player_id	SERIAL	PK
nba_id	INTEGER	UNIQUE; id на NBA.com
first_name	VARCHAR(50)	имя
last_name	VARCHAR(50)	фамилия
birth_date	DATE	дата рождения
nationality	VARCHAR(50)	гражданство
height_cm	SMALLINT	рост (см)
weight_kg	SMALLINT	вес (кг)
position_id	INTEGER	FK → positions
jersey_number	SMALLINT	номер на майке (0–99)
is_active	BOOLEAN	действующий игрок
draft_year	SMALLINT	год драфта
draft_round	SMALLINT	раунд драфта (1 или 2)
draft_pick	SMALLINT	номер выбора на драфте

Таблица 2.5 — Таблица games (матчи)

Атрибут	Тип	Ключ / назначение
game_id	SERIAL	PK
season_id	INTEGER	FK → seasons
home_team_id	INTEGER	FK → teams; команда-хозяин
away_team_id	INTEGER	FK → teams; команда-гость
game_date	DATE	дата матча
home_score	SMALLINT	счёт хозяев
away_score	SMALLINT	счёт гостей
status	VARCHAR(12)	Finished, Scheduled, Postponed
overtime	SMALLINT	число овертаймов (0–6)

Таблица 2.6 — Таблица game_player_stats (факты матча)

Атрибут	Тип	Ключ / назначение
stat_id	SERIAL	PK
game_id	INTEGER	FK → games, ON DELETE CASCADE
player_id	INTEGER	FK → players, ON DELETE RESTRICT
team_id	INTEGER	FK → teams, ON DELETE RESTRICT
minutes_played	DECIMAL(4,1)	минуты на площадке
points	SMALLINT	очки (CHECK ≥ 0)
rebounds_off, rebounds_def	SMALLINT	подборы в нападении и защите
assists, steals, blocks	SMALLINT	передачи, перехваты, блоки
turnovers, fouls	SMALLINT	потери и фолы
fgm, fga, fg3m, fg3a, ftm, fta	SMALLINT	попадания и попытки бросков
plus_minus	SMALLINT	показатель плюс-минус
is_starter	BOOLEAN	признак стартового состава
UNIQUE	—	(game_id, player_id) — гарантирует единственность записи игрока в матче

В таблице game_player_stats пара полей game_id и player_id закреплена ограничением UNIQUE. Это исключает дублирование строк статистики одного игрока в одном матче.

Таблица 2.7 — Таблица player_season_stats (агрегаты за сезон)

Атрибут	Тип	Ключ / назначение
pss_id	SERIAL	PK
player_id	INTEGER	FK → players; часть UNIQUE
season_id	INTEGER	FK → seasons; часть UNIQUE
team_id	INTEGER	FK → teams; часть UNIQUE
games_played	SMALLINT	число игр в сезоне
avg_pts, avg_reb, avg_ast	DECIMAL(5,2)	средние за матч (очки, подборы, передачи)
avg_stl, avg_blk, avg_tov, avg_min	DECIMAL(5,2)	перехваты, блоки, потери, минуты
fg_pct, fg3_pct, ft_pct	DECIMAL(5,4)	проценты попаданий (дробь ∈ [0, 1])
avg_plus_minus	DECIMAL(5,2)	средний плюс-минус
efg_pct, ts_pct, usg_pct	DECIMAL(5,4)	eFG%, TS%, USG (доля 0–1; на UI ×100)
per	DECIMAL(6,2)	PER
bpm	DECIMAL(5,2)	BPM
Ограничение UNIQUE: (player_id, season_id, team_id) — не более одной строки агрегата на игрока, сезон и команду		

Таблица 2.8 — Таблица `player_team_history` (история выступлений)

Атрибут	Тип	Ключ / назначение
<code>history_id</code>	SERIAL	PK
<code>player_id</code>	INTEGER	FK → <code>players</code> ; часть UNIQUE
<code>team_id</code>	INTEGER	FK → <code>teams</code>
<code>season_id</code>	INTEGER	FK → <code>seasons</code>
<code>start_date</code>	DATE	дата начала выступления
<code>end_date</code>	DATE	дата окончания (NULL — по сей день)
<code>contract_type</code>	VARCHAR(20)	Standard, Two-Way, 10-Day и др.

2.1.1 Соответствие нормальным формам

Схема нормализована до третьей нормальной формы (3НФ). Ниже для всех восьми таблиц проверяется выполнение первой (1НФ), второй (2НФ) и третьей (3НФ) нормальных форм [16, с. 390].

Первая нормальная форма (1НФ). В каждом отношении атрибуты атомарны: повторяющихся групп и многозначных полей нет. Статистика игрока в матче хранится отдельной строкой в `game_player_stats`, а не списком показателей в одной ячейке; справочники (`positions`, `teams`) и сущности (`players`, `games`) разнесены по отдельным таблицам.

Вторая нормальная форма (2НФ). Отношение в 2НФ, если оно в 1НФ и каждый неключевой атрибут полностью зависит от всего составного ключа. У таблиц `positions`, `seasons`, `teams`, `players`, `games` первичный ключ простой (`*_id`), поэтому частичных зависимостей быть не может. В `game_player_stats` показатели зависят от пары `game_id` и `player_id`, а не от каждого поля по отдельности. Аналогично в `player_season_stats` агрегаты зависят от составного ключа: `player_id`, `season_id`, `team_id`. В `player_team_history` атрибут `contract_type` относится к конкретному периоду игрока в команде и зависит от составного ключа записи.

Третья нормальная форма (3НФ). Отношение в 3НФ, если оно в 2НФ и для каждой нетривиальной зависимости $X \rightarrow A$ либо X — суперключ, либо A входит в потенциальный ключ. В `teams` поля `city` и `conference` описывают команду напрямую и зависят только от `team_id`. В таблице `players` поля ФИО и физические параметры зависят только от `player_id`, а не от `team_id` (команда задаётся через `player_team_history`). Совпадение данных `player_season_stats` с агрегатами из `game_player_stats` — избыточность между таблицами (витрина для быстрых выборок), а не нарушение 3НФ внутри одного отношения.

Таким образом, все таблицы схемы находятся в 1НФ, 2НФ и 3НФ.

2.1.2 Проектирование представлений

Для обеспечения доступа к данным без прямого обращения к базовым таблицам спроектированы три представления, используемые приложением:

- 1) `v_player_rankings` — объединяет `player_season_stats` и `players`; предоставляет

- сводный рейтинг игроков за сезон по очкам, подборам, передачам и расчётным метрикам;
- 2) `v_team_stats` — агрегирует данные `game_player_stats` в разрезе команды и сезона; реализует функциональное требование расчёта командной статистики без отдельной таблицы фактов;
- 3) `v_top_players` — краткая сводка лидеров по PER для роли `reader` (подмножество `v_player_rankings`).

2.1.3 Проектирование индексов

Для обеспечения производительности аналитических запросов в соответствии с нефункциональными требованиями (раздел 1.3.2) спроектированы следующие индексы [18]:

- 1) `idx_gps_player_id` ON `game_player_stats(player_id)` — ускоряет выборку всех выступлений игрока для агрегации сезонной статистики;
- 2) `idx_gps_team_game` ON `game_player_stats(team_id, game_id)` — поддерживает агрегацию командной статистики в представлении `v_team_stats`;
- 3) `idx_games_season_id` ON `games(season_id)` — ускоряет фильтрацию матчей по сезону;
- 4) `idx_games_date` ON `games(game_date)` — обеспечивает диапазонную фильтрацию по дате проведения матча;
- 5) `idx_pss_season` ON `player_season_stats(season_id)` — ускоряет выборку рейтингов за конкретный сезон;
- 6) `idx_players_name` ON `players(last_name, first_name)` — поддерживает поиск игрока по фамилии.

Ограничение `UNIQUE` на `(game_id, player_id)` в таблице `game_player_stats` автоматически создаёт B-tree индекс, покрывающий запросы поиска по паре «матч – игрок». Все индексы реализованы как B-tree — стандартная структура PostgreSQL, оптимальная для равенства и диапазонных предикатов в аналитических запросах.

2.2 Проектирование ограничений целостности

Помимо стандартных ограничений первичных (PK) и внешних (FK) ключей, в СУБД реализована логика защиты бизнес-правил.

Внешние ключи и каскадное удаление. При удалении матча из `games` связанные строки `game_player_stats` удаляются каскадно (`ON DELETE CASCADE`). Для справочников игроков и команд применяется стратегия `ON DELETE RESTRICT`: физическое удаление запрещено во избежание потери исторической статистики; вместо этого используется логическое удаление посредством флага `is_active`.

ЧЕКСК-ограничения. Обеспечивают согласованность бросков: $fg3m \leq fgm$, $fgm \leq fga$; диапазон фолов от 0 до 6. Для таблицы `games` задано ограничение `CHECK (home_team_id <> away_team_id)`. Для `player_team_history` — ограничение `CHECK (start_date <= end_date)`.

Бизнес-триггеры. На таблице `game_player_stats` реализованы два триггера. Триггер `trg_check_team_score` (AFTER INSERT OR UPDATE, FOR EACH ROW) валидирует равенство суммы очков игроков ($\sum \text{points}$) командному счёту (`home_score` или `away_score`) для завершённых матчей. Триггер `after_game_player_stats_insert` (AFTER INSERT, FOR EACH STATEMENT) вызывает `update_season_stats` для пересчёта витрины. Ограничение по счёту применимо к полностью обработанным данным: игроки, не принявшие участие в матче, в таблицу фактов `game_player_stats` они не вносятся; загружаемые источники должны обеспечивать атрибуцию всех очков конкретным игрокам.

2.3 Проектирование алгоритмов обработки данных

2.3.1 Алгоритмы расчёта метрик игроков

Функции расчёта метрик выполняются на стороне СУБД. При проектировании алгоритма учтено деление на ноль: если знаменатель равен нулю, функция возвращает NULL, что позволяет агрегатным функциям `AVG()` корректно исключить таких игроков из расчёта командной и лиговой статистики без внесения нулевых искажений.

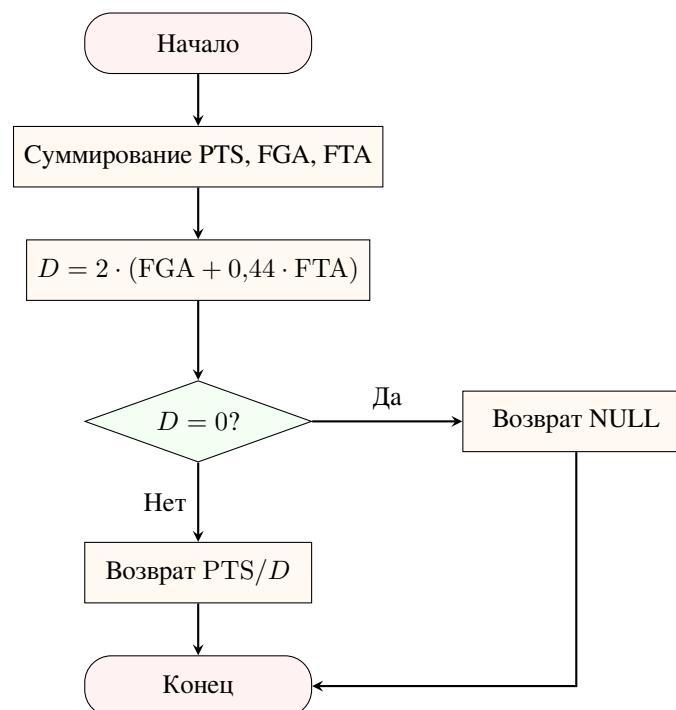


Рисунок 2.2 — Алгоритм функции `calculate_ts`

2.3.2 Алгоритм агрегирования командной статистики

Командная статистика не хранится в отдельной таблице: в соответствии с проектным решением она вычисляется динамически представлением `v_team_stats`. Для каждой пары «команда – сезон» представление суммирует показатели всех игроков из `game_player_stats` по матчам, в которых данная команда выступала в качестве хозяев или гостей, и делит накопленные

суммы на число матчей. Принадлежность строки статистики к хозяевам или гостям определяется через JOIN с таблицей games по полям home_team_id и away_team_id. Результирующий набор содержит средние очки, подборки, передачи, потери и эффективные показатели команды за матч.

2.3.3 Алгоритм массового обновления агрегатов

Процедура update_season_stats пересчитывает витрину player_season_stats для указанного сезона. Алгоритм спроектирован с отказом от курсоров: применяется один наборочный INSERT ... SELECT с последующим ON CONFLICT DO UPDATE. Логика процедуры включает пять шагов:

- 1) агрегация по (player_id, team_id) из game_player_stats за сезон: средние показатели, проценты бросков, отбор строк с $\sum MP \geq 50$;
- 2) для каждой квалифицированной строки вызов calculate_usg, внутри которого командные TmFGA, TmFTA, TmTOV и TmMP суммируются по всем игрокам команды в матчах, где участвовал данный игрок (фильтр по game_id и team_id);
- 3) для calculate_per и calculate_bpm — двухпроходный расчёт: сначала вычисляется ненормализованный показатель игрока, затем среднее по лиге за сезон (только игроки с $\sum MP > 100$), после чего результат нормируется относительно лигового среднего;
- 4) вызов calculate_efg и calculate_ts по сезонным суммам игрока;
- 5) UPSERT в player_season_stats по ключу (player_id, season_id, team_id).

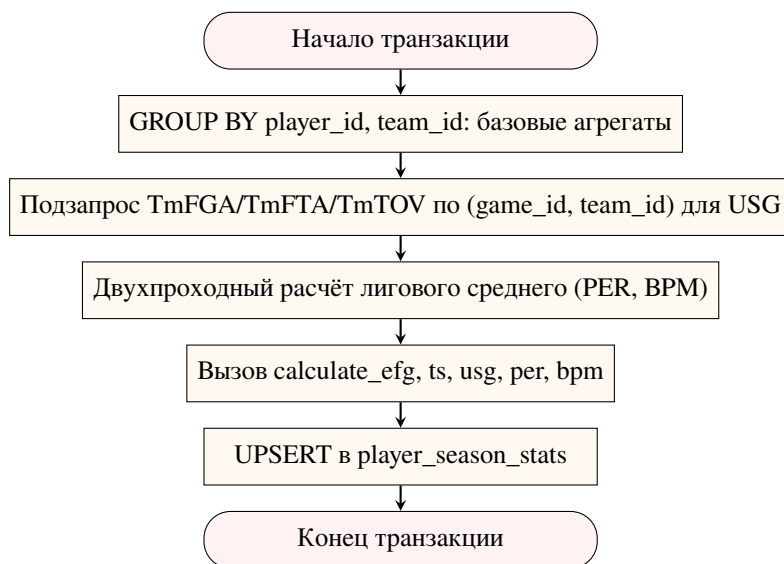


Рисунок 2.3 — Алгоритм процедуры update_season_stats

2.4 Проектирование ролевой модели доступа

В базе данных определены три роли: db_admin, analyst и reader. Администратору выданы полные права на таблицы, последовательности и функции схемы. Аналитику доступны выборка из таблиц и выполнение расчётных функций. Читатель работает исключительно через

представления без прямого доступа к базовым таблицам. Создание и изменение структуры объектов базы данных выполняются от имени выделенного пользователя-владельца схемы при развёртывании. Ограничение доступа роли `reader` к полному представлению `v_player_rankings` и выдача прав только на `v_top_players` обусловлены бизнес-логикой: публичным пользователям доступна лишь краткая сводка лидеров, в то время как полные аналитические выгрузки предназначены только для аналитиков.

Таблица 2.9 — Матрица прав доступа к объектам базы данных

Объект	db_admin	analyst	reader
positions, seasons, teams, players	CRUD	SELECT	—
games, game_player_stats	CRUD	SELECT	—
player_season_stats, player_team_history	CRUD	SELECT	—
Функции <code>calculate_*</code>	EXECUTE	EXECUTE	—
v_team_stats, v_top_players	—	SELECT	SELECT
v_player_rankings	SELECT	SELECT	—

2.5 Проектирование программного обеспечения и кэширования

2.5.1 Архитектура приложения

Программное обеспечение реализует трёхзвенную архитектуру. Уровень представления — веб-клиент, формирующий HTTP-запросы к серверу. Уровень приложения — REST API-сервер, выполняющий маршрутизацию запросов, проверку авторизации и вызов процедур СУБД. Уровень данных — реляционная СУБД и in-мемори кэш, взаимодействие которых описано в подразделе 2.5.3.

API организован в пять групп маршрутов (`players`, `teams`, `stats`, `league`, `admin`), соответствующих вариантам использования из раздела 1.5.2. Итого спроектировано 20 бизнес-маршрутов.

2.5.2 Спецификация REST API

Сезон в параметрах запроса передаётся целочисленным `season_id` (внешний ключ таблицы `seasons`); метка `label` (например, 2023–24) возвращается в ответах API для отображения в интерфейсе. Список сезонов клиент получает через `GET /league/seasons`.

Таблица 2.10 — Маршруты группы /players

Метод	Путь	Параметры	Вариант исп.
GET	/players	season_id (обяз.), position, team_id, search, limit, offset	Просмотр статистики
GET	/players/{player_id}	—	Просмотр статистики
GET	/players/{player_id}/stats	season_id (опц.)	Просмотр статистики
GET	/players/{player_id}/gamelog	season_id (обяз.)	Просмотр статистики
GET	/players/{player_id}/career	—	Просмотр статистики
GET	/players/{player_id}/teams	—	Просмотр статистики

Таблица 2.11 — Маршруты группы /teams

Метод	Путь	Параметры	Вариант исп.
GET	/teams	—	Просмотр статистики
GET	/teams/{team_id}	—	Просмотр статистики
GET	/teams/{team_id}/roster	season_id (обяз.)	Просмотр статистики
GET	/teams/{team_id}/games	season_id (опц.)	Просмотр статистики

Таблица 2.12 — Маршруты группы /stats

Метод	Путь	Параметры	Вариант исп.
GET	/stats/leaders	metric, season_id (обяз.), team_id, position, min_games, limit	Просмотр статистики
GET	/stats/advanced	season_id (обяз.)	Расчёт метрик
GET	/stats/scatter	season_id (обяз.)	Расчёт метрик
GET	/stats/boxscore/{game_id}	—	Просмотр статистики
GET	/stats/compare/players	p1_id, p2_id, season_id (обяз.)	Просмотр статистики

Параметр `metric` в маршруте `/stats/leaders` принимает одно из значений: `avg_pts`, `avg_reb`, `avg_ast`, `avg_stl`, `avg_blk`, `per`, `ts_pct`, `efg_pct`, `bpm` и `usg_pct`.

Таблица 2.13 — Маршруты группы /league

Метод	Путь	Параметры	Вариант исп.
GET	/league/seasons	—	Просмотр статистики
GET	/league/dashboard	season_id (обяз.)	Просмотр статистики
GET	/league/trends	—	Просмотр статистики
GET	/league/search	q (обяз., мин. 2 символа)	Просмотр статистики

Маршрут /league/seasons возвращает перечень доступных сезонов. Ответ содержит поля season_id, label, season_type, start_date и end_date; клиент использует его для формирования элементов выбора сезона. Маршрут /league/search реализует полнотекстовый поиск или поиск по подстроке (ILIKE) по полям first_name и last_name таблицы players, а также name таблицы teams, возвращая комбинированный массив совпадений.

Таблица 2.14 — Маршруты группы /admin

Метод	Путь	Параметры	Вариант исп.
POST	/admin/refresh/{season_id}	заголовок X-API-Key	Управление данными

Разграничение доступа на уровне API реализовано для группы /admin: маршрут проверяет наличие корректного значения в заголовке X-API-Key. Разграничение ролей analyst и reader обеспечивается средствами СУБД в соответствии с матрицей прав (таблица 2.9): API-сервер устанавливает соединение с базой данных от имени пользователя с соответствующим набором привилегий.

2.5.3 Кэширование результатов запросов

Для обеспечения высокой пропускной способности спроектирован паттерн Cache-Aside с использованием in-memory хранилища и API-сервера.

Алгоритм: приложению поступает REST-запрос. Сервер проверяет наличие ключа в кэше. При попадании данные возвращаются из кэша. При промахе сервер запрашивает реляционную БД, сериализует ответ, сохраняет его в кэш с заданным временем жизни и возвращает клиенту.

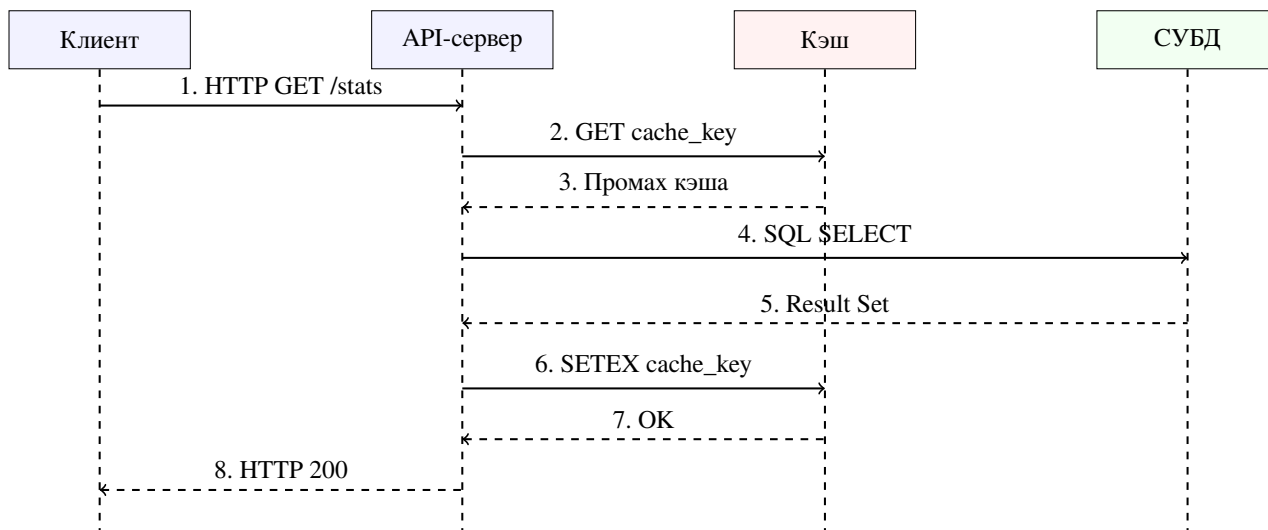


Рисунок 2.4 — UML-диаграмма последовательности при паттерне Cache-Aside

Инвалидация кэша выполняется по шаблонам ключей лидербордов, дашборда и данных команд только для изменённого сезона, а не полным сбросом кэша: это предотвращает одновременную инвалидацию всех ключей и снижает риск пиковой нагрузки на СУБД при массовом промахе кэша — эффекта «шторма кэша», который возникает, когда множество параллельных запросов одновременно обнаруживают истёкший ключ и нагружают базу данных.

Вывод

Спроектирована реляционная схема из 8 таблиц, 3 представлений и 6 ключевых индексов; показано соответствие всех таблиц 1НФ, 2НФ и 3НФ. Бизнес-логика метрик реализована SQL-процедурами на стороне СУБД; ограничения целостности закреплены на уровне CHECK-ограничений и триггера. Определена ролевая модель из трёх ролей с разграничением прав на уровне таблиц, функций и представлений. Спроектировано REST API из 20 маршрутов, организованных в пять групп и покрывающих все три варианта использования системы. Для снижения задержки аналитических выборок спроектировано кэширование по паттерну Cache-Aside с префиксной инвалидацией.

3 Технологический раздел

3.1 Выбор и обоснование средств реализации

Профиль нагрузки проектируемой системы носит гибридный характер (HTAP): преобладают аналитические запросы на чтение с агрегацией, при этом процедура массового обновления статистики требует транзакционной целостности.

3.1.1 СУБД

В качестве основной СУБД выбрана PostgreSQL 15 [18]. Её использование обосновано следующими факторами:

- наличие встроенного процедурного языка PL/pgSQL, необходимого для реализации математических алгоритмов расчёта продвинутых метрик;
- поддержка атомарных операций слияния строк (конструкция `INSERT ... ON CONFLICT DO UPDATE`) для эффективного пересчёта сезонных агрегатов;
- возможность декларативного обеспечения целостности данных с помощью CHECK-ограничений на уровне DDL.

3.1.2 Кэширование

Паттерн Cache-Aside реализован на базе in-memory СУБД Redis 7 [17]. Выбор обоснован поддержкой операций сканирования и группового удаления по шаблону, что необходимо для инвалидации связанных ключей при пересчёте статистики отдельного сезона. Команда SETEX позволяет атомарно устанавливать значение и время жизни ключа, исключая состояние гонки.

3.1.3 Серверный слой

REST API реализован на Python 3.11 с фреймворком FastAPI [19]. Фреймворк построен на стандарте ASGI и модели асинхронного ввода-вывода: каждый обработчик маршрута является корутиной, что позволяет обслуживать конкурентные запросы в одном процессе без блокировки потока. Механизм внедрения зависимостей позволяет декларативно привязать к маршруту сессию конкретной роли СУБД, что соответствует ролевой модели из раздела 2.4: разграничение привилегий происходит на уровне соединения с PostgreSQL, а не в логике приложения. Для взаимодействия с СУБД применяется драйвер asynpg [20], реализующий бинарный протокол PostgreSQL, в связке с SQLAlchemy 2.0 для конструирования динамических аналитических запросов.

3.1.4 Клиентский слой

Клиентская часть реализована в виде одностраничного приложения (SPA) с использованием библиотеки React 18 [21] и языка TypeScript. Строгая типизация моделей данных снижает риск ошибок при взаимодействии с API. Для визуализации статистики применена библиотека Recharts [22], поддерживающая построение линейных, столбчатых, радарных диаграмм и диаграмм рассеяния.

3.2 Физическая реализация базы данных

3.2.1 Создание таблиц и ограничений

Физическая реализация структуры базы данных выполнена с помощью DDL-скриптов (полный листинг приведён в Приложении А). В листинге 3.1 представлен фрагмент создания таблицы фактов `game_player_stats`, иллюстрирующий механизм декларативного обеспечения ссылочной и логической целостности.

Listing 3.1 — DDL таблицы `game_player_stats` (фрагмент)

```
CREATE TABLE game_player_stats (  
    stat_id    SERIAL PRIMARY KEY,  
    game_id    INTEGER NOT NULL  
                REFERENCES games(game_id) ON DELETE CASCADE,  
    player_id  INTEGER NOT NULL  
                REFERENCES players(player_id) ON DELETE RESTRICT,  
    team_id    INTEGER NOT NULL  
                REFERENCES teams(team_id) ON DELETE RESTRICT,  
    points     SMALLINT NOT NULL DEFAULT 0 CHECK (points >= 0),  
    fouls      SMALLINT NOT NULL DEFAULT 0 CHECK (fouls BETWEEN 0  
AND 6),  
    CONSTRAINT uq_game_player UNIQUE (game_id, player_id)  
);
```

3.2.2 Реализация индексов

Аналитические индексы (Приложение А) реализованы на базе структуры В-дерева. В листинге 3.2 продемонстрировано создание ключевых индексов для таблицы фактов.

Listing 3.2 — Создание индексов аналитических запросов (фрагмент)

```
CREATE INDEX idx_gps_player_id  
    ON game_player_stats(player_id);  
CREATE INDEX idx_gps_team_game  
    ON game_player_stats(team_id, game_id);
```

```
CREATE INDEX idx_pss_season
ON player_season_stats(season_id, per DESC NULLS LAST);
```

Исследование эффективности данных структур и их влияния на время выполнения запросов приведено в исследовательском разделе (глава 4).

3.2.3 Загрузка тестовых данных

Первоначальное наполнение базы данных выполнено ETL-скриптом `scripts/load_data.py` на языке Python. Скрипт извлекает архивную статистику NBA за пять регулярных сезонов (2019/20–2023/24) из официального портала NBA Stats [1] через библиотеку `nba_api` [23], формирующую HTTP-запросы к сервису `stats.nba.com`.

Полученные данные нормализуются и загружаются в реляционные таблицы PostgreSQL пакетами в порядке, соответствующем иерархии внешних ключей: справочники, затем игроки, матчи, статистика матчей, история команд. Использование официального источника сохраняет естественное распределение показателей реальных игроков, что критически важно для корректной верификации относительных метрик.

По результатам контрольных запросов `SELECT COUNT(*)` объём ключевых таблиц после загрузки приведён в таблице 3.1. Таблица фактов `game_player_stats` содержит 150 629 записей, таблица `games` — 5 829 матчей. Это полностью покрывает нефункциональное требование об объёме данных.

Таблица 3.1 — Объём загруженных данных (контрольный запрос `SELECT COUNT(*)`)

Таблица	Число строк
<code>game_player_stats</code>	150 629
<code>games</code>	5 829
<code>players</code>	5 126
<code>player_season_stats</code>	2 813
<code>teams</code>	30

3.3 Реализация логики предметной области в СУБД

3.3.1 Хранимые функции расчёта метрик

Логика расчёта спортивных метрик реализована в виде хранимых PL/pgSQL-функций (Приложение Б). В листинге 3.3 приведён фрагмент функции `calculate_ts`, включающий программную защиту от деления на ноль.

Listing 3.3 — Реализация функции `calculate_ts` (фрагмент)

```
CREATE OR REPLACE FUNCTION calculate_ts(
    p_player_id INT, p_season_id INT, p_team_id INT DEFAULT NULL
) RETURNS DECIMAL(5,4) LANGUAGE plpgsql STABLE AS $$
```

```

DECLARE
    v_pts NUMERIC; v_fga NUMERIC; v_fta NUMERIC; v_denom NUMERIC;
BEGIN
    SELECT COALESCE(SUM(gps.points), 0),
           COALESCE(SUM(gps.fga), 0),
           COALESCE(SUM(gps.fta), 0)
    INTO v_pts, v_fga, v_fta
    FROM game_player_stats gps
    JOIN games g ON g.game_id = gps.game_id
    WHERE gps.player_id = p_player_id
           AND g.season_id = p_season_id
           AND (p_team_id IS NULL OR gps.team_id = p_team_id);

    IF v_fga = 0 AND v_fta = 0 THEN RETURN NULL; END IF;

    v_denom := 2.0 * (v_fga + 0.44 * v_fta);
    IF v_denom = 0 THEN RETURN NULL; END IF;

    RETURN ROUND(v_pts::DECIMAL / v_denom, 4);
END; $$;

```

Все расчётные функции помечены квалификатором `STABLE`, что позволяет оптимизатору запросов кэшировать результаты вызовов в рамках одного оператора. Агрегация сезонной статистики выполняется хранимой процедурой `update_season_stats` с использованием атомарной конструкции `INSERT ... ON CONFLICT DO UPDATE`.

3.3.2 Триггеры логической целостности

Для защиты данных от аномалий реализован триггер `trg_check_team_score`. Он проверяет бизнес-правило предметной области: сумма очков всех игроков команды в конкретном матче должна равняться итоговому командному счёту.

Listing 3.4 — Триггер проверки корректности очков (фрагмент)

```

CREATE OR REPLACE FUNCTION trg_validate_team_score_row()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_sum INT; v_expected INT;
BEGIN
    IF v_expected IS NOT NULL AND v_sum <> v_expected THEN
        RAISE EXCEPTION
            'Player points sum (%) does not match team score (%)',
            v_sum, v_expected;
    END IF;
END;

```

```
END IF;  
RETURN COALESCE(NEW, OLD);  
END; $$;
```

3.4 Реализация ролевой модели доступа

Настройка прав доступа (листинг 3.5) выполнена в соответствии с принципом минимальных привилегий. Роль публичного доступа (`reader`) лишена возможности прямого обращения к базовым таблицам; чтение осуществляется исключительно через защищённые представления.

Listing 3.5 — Разграничение доступа командами привилегий

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst;  
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO analyst;  
  
GRANT SELECT ON v_team_stats TO reader;  
GRANT SELECT ON v_top_players TO reader;
```

Корректность ограничений для роли `reader` проверена подключением к БД от её имени: запрос `SELECT * FROM game_player_stats` завершился ошибкой `permission denied` на уровне PostgreSQL, тогда как `SELECT` из представлений `v_team_stats` и `v_top_players`, перечисленных в листинге 3.5, выполняется успешно. Аналитические маршруты API, в том числе `GET /teams/standings`, открывают сессию под ролью `analyst` и для сводной командной статистики обращаются к представлению `v_team_stats`. У роли `analyst` есть `SELECT` к базовым таблицам (табл. 2.9), поэтому представления для неё служат удобными витринами, а не единственным способом чтения данных.

3.5 Реализация программного обеспечения и интерфейса

3.5.1 Серверное приложение

Серверное приложение реализовано в соответствии со спецификацией REST API конструкторского раздела (подраздел 2.5.2). Точкой входа служит `app/main.py`, регистрирующий пять групп маршрутов и CORS-middleware. Подключение к PostgreSQL организовано через три независимых пула соединений — по одному на каждую роль СУБД. Привязка маршрута к роли осуществляется через аргумент `Depends`:

Listing 3.6 — Привязка маршрута к роли СУБД

```
@router.get("/leaders")  
async def get_leaders(  
    season_id: int,  
    db: AsyncSession = Depends(get_db_analyst),  
    cache: CacheManager = Depends(get_cache),
```

):

Маршрут `/stats/leaders` открывает соединение от имени роли `analyst`, имеющей право `SELECT` к агрегированной статистике и `EXECUTE` к функциям расчёта метрик; прямое изменение базовых таблиц по-прежнему запрещено. Жизненный цикл сессии управляется генератором: при любом исходе обработчика сессия закрывается, а незафиксированные изменения откатываются.

3.5.2 Кэширование

Паттерн `Cache-Aside` реализован классом `CacheManager`. Структура каждого кэшируемого маршрута единообразна: вычисляется ключ из параметров запроса, выполняется попытка чтения из `Redis`; при промахе — SQL-запрос к `PostgreSQL`, запись результата в кэш с заданным временем жизни и возврат клиенту. Пример приведён в листинге 3.7.

Listing 3.7 — `Cache-Aside` в маршруте `/stats/leaders`

```
cache_key = (
    f"leaders:{metric}:{season_id}:{team_id}:{position}:{min_games}
    {limit}"
)
cached = await cache.get(cache_key)
if cached:
    return cached
result = await db.execute(query)
data = [...]
await cache.set(cache_key, data, ttl=900)
return data
```

Время жизни ключей дифференцировано по типу данных: данные завершённых матчей кэшируются на один год, лидерборды и ростеры — на 15 минут, результаты поиска — на 2 минуты. Инвалидация при вызове `POST /admin/refresh/{season_id}` выполняется по шаблону ключей затронутого сезона в соответствии с проектным решением подраздела 2.5.3.

3.5.3 Пользовательский интерфейс

Клиентское одностраничное приложение реализовано на `React 18` с `TypeScript`. Все типы моделей данных описаны в `src/types/index.ts` и используются совместно компонентами и функциями API-клиента.

Приложение включает пять разделов навигации:

- 1) `Dashboard` — KPI-блок лиги, топ-5 игроков по `PER`, диаграмма рассеяния «очки vs `eFG%`», линейные графики трендов по сезонам, горизонтальные гистограммы топ-10 команд и игроков по `USG%`;

- 2) Players — таблица игроков с серверными фильтрами по позиции, команде и имени, пагинацией; клик открывает модальное окно профиля;
- 3) Teams — турнирная таблица по конференциям (Восток/Запад) за выбранный сезон (W, L, W%, GB); переход на страницу команды с ростером;
- 4) Leaderboard — рейтинг по десяти метрикам; клик открывает профиль игрока;
- 5) Advanced Stats — диаграммы рассеяния «USG% vs TS%» и «PER vs среднее время на площадке».

Сравнение двух игроков вынесено в модальное окно: из профиля игрока кнопкой «добавить к сравнению» формируется список из двух участников; плавающая панель в углу экрана открывает окно с совмещённым радарным графиком, гистограммой базовой статистики и таблицей метрик с выделением лучшего значения (данные — GET /stats/compare/players).

Внешний вид раздела Dashboard представлен на рисунке 3.1.



Рисунок 3.1 — Главная страница приложения (Dashboard)

Модальное окно игрока доступно из любого раздела и отображает демографические данные, бейджи базовых и продвинутых метрик за выбранный сезон, радарный график характеристик по шести осям и игровой журнал последних 20 матчей (рисунок 3.2). Кнопка добавления в сравнение доступна в шапке модального окна.

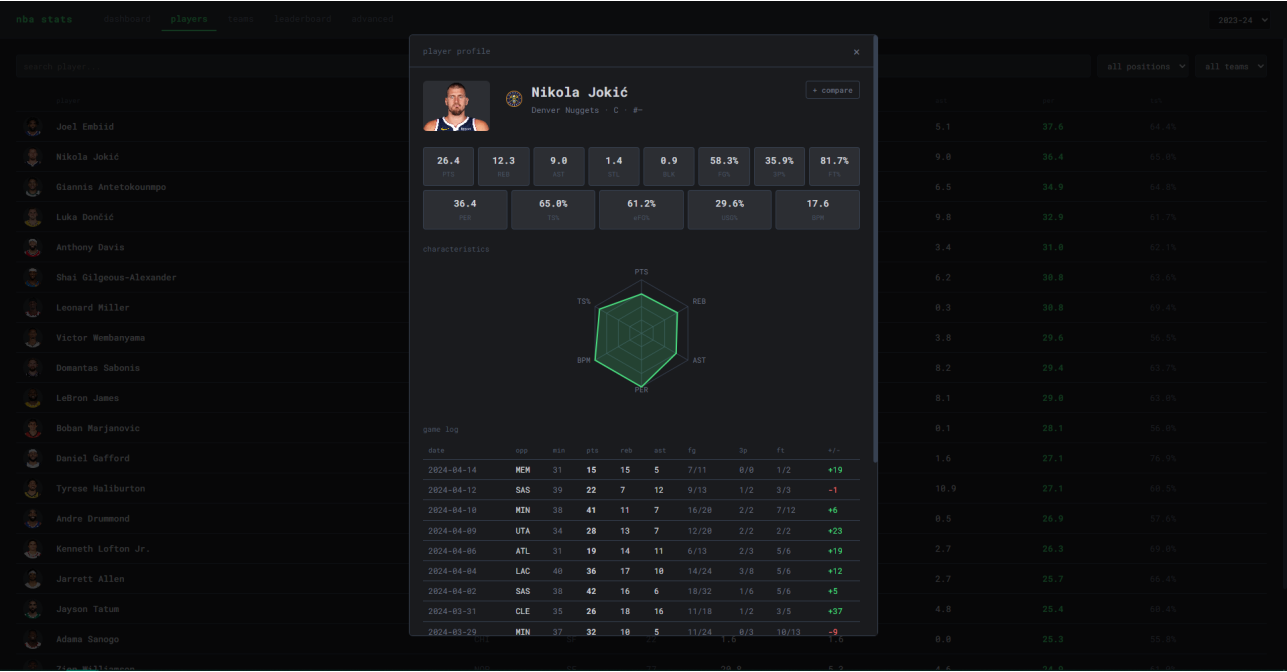


Рисунок 3.2 — Модальное окно профиля игрока: сезонные метрики и игровой журнал

Модальное окно сравнения двух выбранных игроков представлено на рисунке 3.3.

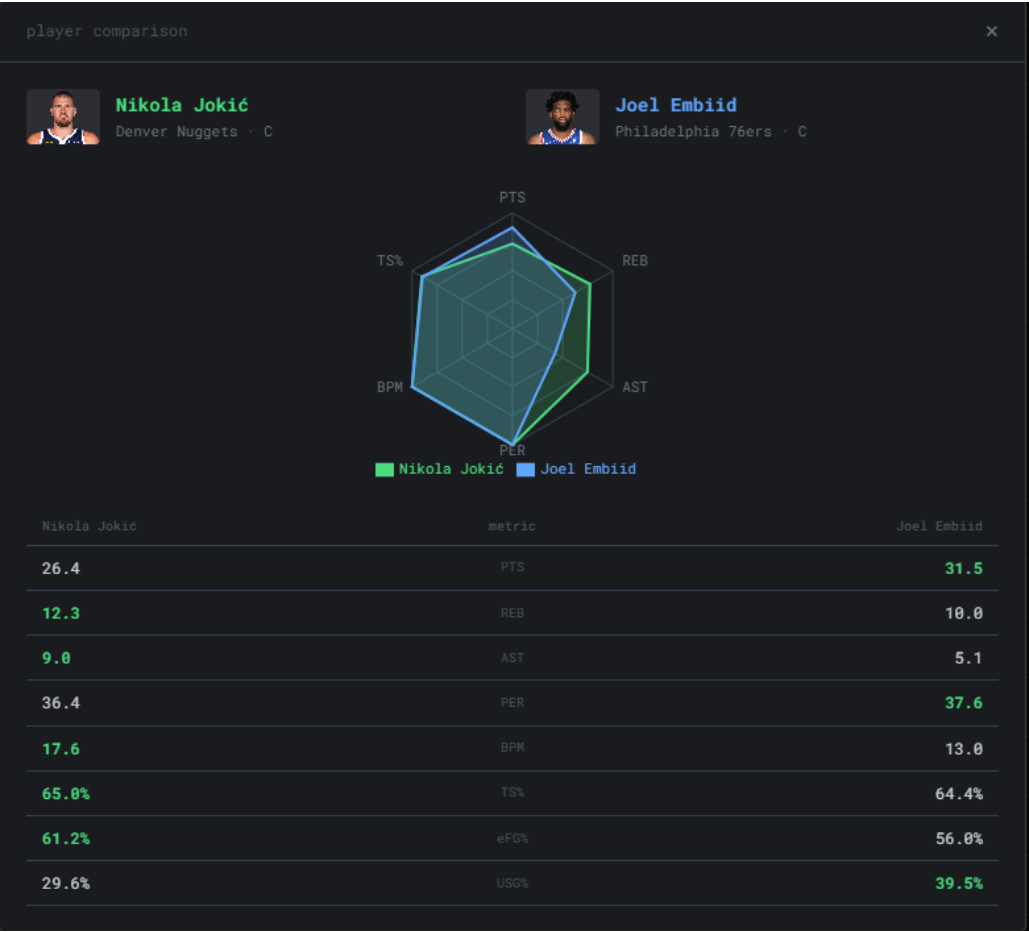


Рисунок 3.3 — Модальное окно сравнения профилей двух игроков

3.6 Тестирование и верификация аналитических данных

Тестирование логики СУБД проводилось методом чёрного ящика. Для проверки точности математических агрегаций выполнен регрессионный тест на реальных данных из загруженной базы: эталонные значения получены независимым расчётом по формулам раздела 1.4, после чего сопоставлены с результатами хранимых функций PostgreSQL.

В качестве тестового объекта выбран профиль игрока Nikola Jokić (`player_id = 2327`) за регулярный сезон 2023-24 (`season_id = 5`). Прямой SQL-запрос суммы сырых фактов из таблицы `game_player_stats` вернул следующие значения:

- очки: 2085;
- броски с игры: 1411;
- штрафные попытки: 438.

Ручной расчёт истинного процента бросков по формуле (1.2):

$$TS\% = \frac{2085}{2 \cdot (1411 + 0,44 \cdot 438)} = \frac{2085}{3207,44} \approx 0,6501.$$

Для верификации в СУБД выполнен вызов хранимой функции:

```
SELECT calculate_ts(2327, 5);
```

Результат совпал с ручным расчётом. Аналогичная верификация проведена для `eFG%` и `USG%`; расхождений не выявлено.

Верификация `PER` выполнена по той же методике. Из таблицы `game_player_stats` получены сезонные суммы для `player_id = 2327`: `PTS = 2085`, `TRB = 976`, `AST = 708`, `STL = 108`, `BLK = 68`, `TOV = 237`, `PF = 194`, `FGM/FGA = 822/1411`, `FTM/FTA = 358/438`, `MP = 2736,4`. Ненормализованный вклад по формуле `uPER` (раздел 1.4):

$$uPER = (2085 + 976 + 708 + 108 + 68 - 237 - 194 - 589 - 35,2) \cdot \frac{48}{2736,4} = 2889,8 \cdot \frac{48}{2736,4} \approx 50,69.$$

Лиговое среднее по формуле (1.5) по 475 игрокам с `MP > 100`: `lg_aPER = 20,88`. Итоговый рейтинг по формуле (1.6):

$$PER = 50,69 \cdot \frac{15}{20,88} \approx 36,41.$$

Вызов `SELECT calculate_per(2327, 5)` вернул `36,41` — расхождений не выявлено.

Тестирование граничных случаев подтвердило корректный возврат значения `NULL` при отсутствии попыток бросков и нулевом времени на площадке. Работа триггера `trg_check_team_score` проверена попыткой вставки строки с намеренно неверной суммой очков: триггер поднял исключение `Player points sum does not match team score`, вставка была отклонена — поведение соответствует ожидаемому (таблица 3.2).

Таблица 3.2 — Контрольные тест-кейсы функций расчёта метрик

№	Сценарий	Входные данные	Ожидание	Результат
1	Нормальный профиль	<code>calculate_ts(2327, 5)</code>	$\approx 0,6501$	совпало
2	Нет попыток броска	<code>calculate_ts(0, 0)</code>	NULL	NULL
3	Нулевые FGA (eFG%)	<code>calculate_efg(0, 0)</code>	NULL	NULL
4	Пересчёт витрины	<code>CALL update_season_stats(5)</code>	обновлены строки pss	выполнено
5	Верификация PER	<code>calculate_per(2327, 5)</code>	$\approx 36,41$ (формулы 1.4–1.6)	36,41, совпало
6	Триггер целостности счёта	<code>INSERT</code> с суммой очков, не совпадающей с <code>home_score</code>	EXCEPTION	исключение поднято, вставка отклонена

Вывод

В технологическом разделе обоснован стек технологий (PostgreSQL 15, Redis 7, FastAPI, React 18) и реализована физическая структура базы данных. Разработаны PL/pgSQL-функции и триггеры, обеспечивающие расчёт метрик и консистентность данных. С помощью написанного ETL-скрипта БД наполнена реальной статистикой NBA (более 150 тыс. фактов). Успешно реализованы механизмы кэширования (Cache-Aside) и ролевого доступа. Регрессионное тестирование на реальных данных подтвердило математическую корректность работы базы данных и хранимых процедур.

4 Исследовательский раздел

4.1 Цель и методика исследования

Целью раздела является количественная оценка влияния объёма данных, стратегий индексации и уровня параллельной нагрузки на производительность аналитических запросов, а также сравнение прямой агрегации по таблице фактов с обращением к предвычисленной витрине и замер эффективности кэширования Redis.

Измерения выполнены скриптом Python 3.11 с библиотеками `asyncpg` (прямые SQL-запросы) и `httpx` (HTTP-запросы к API). Стенд: СУБД PostgreSQL 15 и API-сервер развёрнуты в изолированных Docker-контейнерах на одном хосте, что исключает сетевые задержки. Для каждого сценария: 10 прогревочных итераций (не включаются в статистику), 100 рабочих итераций (30 для холодного старта кэша). Метрики: медиана, среднее, 95-й перцентиль (P95).

4.2 Влияние объёма данных и стратегий индексации

4.2.1 Влияние объёма данных на время выполнения запроса

Запрос к витрине `player_season_stats` (топ-20 по PER) измерялся при наличии составного индекса (`season_id`, `per` DESC) [18].

Таблица 4.1 — Время выполнения запроса в зависимости от объёма витрины

Объём данных	Строк	Медиана (мс)	Среднее (мс)	P95 (мс)
1 сезон	522	0,24	0,26	0,38
3 сезона	1 681	0,26	0,26	0,35
5 сезонов	2 813	0,24	0,24	0,30

Медиана стабильна в диапазоне 0,24–0,26 мс при росте числа строк в 5,4 раза. Это подтверждает логарифмическую сложность $O(\log N)$ B-tree индекса: выборка топ-20 обходит $O(\log N + 20)$ узлов дерева независимо от общего объёма таблицы.

4.2.2 Исследование стратегий индексации

Сравнивались четыре конфигурации индексов на витрине `player_season_stats` (2 813 строк, `season_id` = 5).

Listing 4.1 — Запрос для сравнения стратегий индексирования

```
SELECT player_id, per, avg_pts
FROM player_season_stats
WHERE season_id = 5
ORDER BY per DESC NULLS LAST
LIMIT 20;
```

Таблица 4.2 — Влияние конфигурации индексов на производительность запроса

Конфигурация индекса	Медиана (мс)	Среднее (мс)	P95 (мс)
1. Без индексов (Seq Scan)	0,64	0,80	1,49
2. B-tree по (season_id)	0,55	0,64	1,09
3. Составной (season_id, per DESC)	0,28	0,30	0,47
4. Частичный WHERE per IS NOT NULL	0,55	0,58	0,75

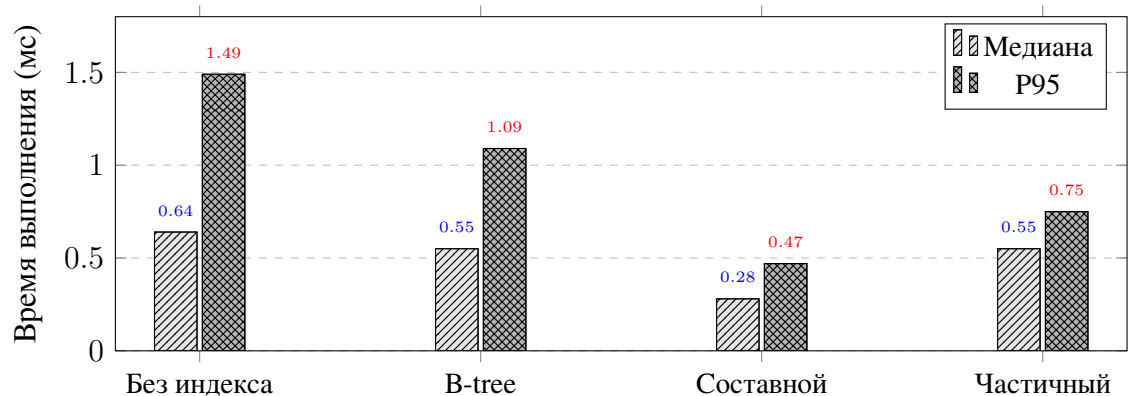


Рисунок 4.1 — Время выполнения запроса при различных стратегиях индексирования

Простой индекс по `season_id` (конфигурация 2) снижает медиану на 14% относительно Seq Scan: планировщик использует Index Scan, но выполняет явную сортировку в памяти. Составной индекс (конфигурация 3) кодирует порядок `per DESC` непосредственно в B-дереве, исключая шаг Sort; медиана снижается до 0,28 мс — на 56% лучше Seq Scan и на 49% лучше простого индекса. Частичный индекс (конфигурация 4) эквивалентен простому: предикат `per IS NOT NULL` отсеивает менее 1% строк, поэтому оптимизатор отвергает его как неселективный.

4.3 Сравнение витрины и прямой агрегации по таблице фактов

Третий эксперимент сопоставляет обращение к предвычисленной витрине `player_season_stats` с прямым агрегирующим запросом к таблице фактов `game_player_stats`. Объем данных для сезона 5: 32 382 строки фактов, 580 строк витрины.

Тестируемый запрос прямой агрегации:

Listing 4.2 — Прямой агрегирующий запрос к таблице фактов

```
SELECT gps.player_id ,
       SUM(gps.points)      AS total_pts ,
       COUNT(*)             AS games ,
       AVG(gps.points::float) AS avg_pts
FROM game_player_stats gps
```

```

JOIN games g ON g.game_id = gps.game_id
WHERE g.season_id = 5
GROUP BY gps.player_id
ORDER BY total_pts DESC
LIMIT 20;

```

Таблица 4.3 — Время выполнения прямой агрегации при различных индексах и сравнение с витриной

Конфигурация	Медиана (мс)	Среднее (мс)	P95 (мс)
Прямая агрегация: без индексов	46,81	56,44	145,26
+ idx_games_season_id	20,55	24,18	38,76
+ idx_gps_player_id	19,66	21,13	32,44
+ idx_gps_team_game	30,24	54,24	145,99
Витрина (составной индекс)	0,32	0,33	0,42

Индексы `idx_games_season_id` и `idx_gps_player_id` снижают медиану прямой агрегации примерно вдвое — до 19–21 мс — за счёт сокращения числа блоков, читаемых при JOIN. Добавление составного индекса (`team_id`, `game_id`) ухудшает результат: медиана возрастает до 30,24 мс, P95 — до 146 мс. Это объясняется выбором планировщика: индекс (`team_id`, `game_id`) неселективен для запроса без предиката по `team_id`, поэтому оптимизатор выбирает менее эффективный план с Bitmap Index Scan вместо хэш-агрегации. Результат демонстрирует, что избыточные индексы способны ухудшать производительность.

Витрина обеспечивает медиану 0,32 мс — в 61 раз быстрее лучшего варианта прямой агрегации (19,66 мс) и в 146 раз быстрее запроса без индексов. Прирост обусловлен исключением JOIN и GROUP BY: витрина хранит готовые агрегаты, запрос к ней сводится к фильтрации по индексу с прямым чтением результата.

4.4 Исследование эффективности кэширования

Четвёртый эксперимент измеряет сквозное время ответа маршрута GET /stats/leaders?metric=per&season_id=5&limit=20 в двух сценариях.

Cache Miss. Перед каждой из 30 итераций кэш сбрасывался командой FLUSHDB; запрос полностью обрабатывался PostgreSQL.

Cache Hit. Кэш прогревался одним запросом; последующие 100 итераций обслуживались Redis без обращения к СУБД.

Таблица 4.4 — Сравнение времени ответа API: Cache Miss и Cache Hit

Источник данных	Медиана (мс)	Среднее (мс)	P95 (мс)
PostgreSQL (Cache Miss)	14,38	14,83	20,07
Redis (Cache Hit)	3,08	3,40	5,11

Кэширование даёт ускорение в 4,7 раза по медиане (14,38 мс → 3,08 мс). Оба сценария укладываются в нефункциональное требование 200 мс (раздел 1.3.2). Разница обусловлена исключением SQL-выполнения и сериализации: при Cache Hit API выполняет только десериализацию JSON из Redis и HTTP-ответ.

4.5 Исследование производительности при параллельной нагрузке

Пятый эксперимент оценивает деградацию при конкурентном доступе. Перед каждым раундом кэш прогревался одним запросом. Нагрузка генерировалась `asyncio.gather` с уровнями 1, 10, 50 и 100 одновременных подключений (10 раундов на уровень).

Таблица 4.5 — Зависимость времени ответа API от уровня конкурентной нагрузки

Параллельных запросов	Медиана (мс)	Среднее (мс)	P95 (мс)	Ошибки (%)
1	2,55	3,01	4,76	0
10	13,98	15,08	23,99	0
50	81,24	85,66	159,05	0
100	183,10	190,02	331,45	0

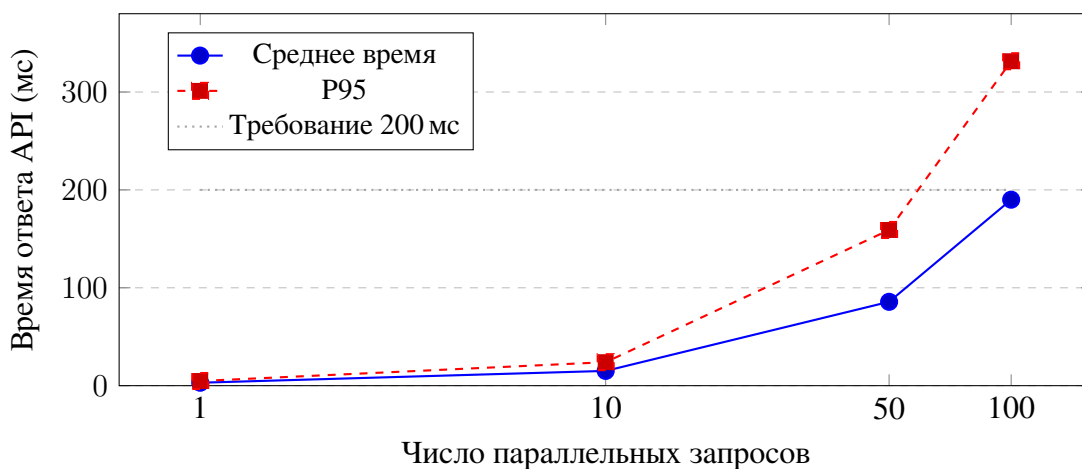


Рисунок 4.2 — Деградация производительности API при росте параллельной нагрузки

При нагрузке до 50 запросов P95 (159,05 мс) укладывается в требование 200 мс. При 100 подключениях медиана (183,10 мс) остаётся в пределах требования, однако P95 достигает 331,45 мс, что свидетельствует о выбросах в хвосте распределения. Ошибок HTTP 5xx не зафиксировано ни при одном уровне нагрузки. Деградация обусловлена насыщением пула соединений с PostgreSQL и очереди event loop единственного экземпляра Uvicorn.

4.6 Анализ результатов и рекомендации

1. Витрина как основной инструмент производительности.

Прямая агрегация 32 382 строк фактов с оптимальными индексами занимает 19,66 мс; обраще-

ние к витрине — 0,32 мс (ускорение в 61 раз). Предвычисленная витрина `player_season_stats` является ключевым архитектурным решением для выполнения требования 200 мс на аналитических запросах.

2. Составной индекс исключает шаг сортировки.

Составной индекс (`season_id`, `per DESC`) снижает медиану на 56% относительно Seq Scan, кодируя порядок сортировки в структуре B-дерева. Частичный индекс с неселективным предикатом эквивалентен простому и не рекомендуется. Избыточный составной индекс (`team_id`, `game_id`) при агрегации без предиката по `team_id` ухудшает производительность — планировщик выбирает менее эффективный план; индекс следует применять только при наличии соответствующего предиката в запросе.

3. Кэширование Redis обязательно при конкурентном доступе.

Cache-Aside обеспечивает ускорение в 4,7 раза (14,38 мс → 3,08 мс). При нагрузке 50 запросов без кэша медиана составила бы 85 мс с накоплением на СУБД; кэш снижает её до единиц миллисекунд.

4. Предел одного экземпляра — 50 конкурентных запросов.

Система гарантированно выполняет требование $P95 < 200$ мс при нагрузке до 50 подключений. При 100 подключениях медиана (183 мс) остаётся в пределах требования, но $P95$ (331 мс) превышает его. Для пиковых нагрузок рекомендуется горизонтальное масштабирование: несколько экземпляров API-сервера за reverse-прокси (Nginx, round-robin) с единым кэшем Redis.

Вывод

Проведено пять нагрузочных экспериментов. Предвычисленная витрина `player_season_stats` ускоряет аналитические выборки в 61 раз относительно прямой агрегации по таблице фактов с оптимальными индексами. Составной индекс (`season_id`, `per DESC`) на витрине обеспечивает стабильное время отклика 0,24–0,26 мс при пятикратном росте объёма данных. Кэширование результатов в Redis снижает медианную задержку API в 4,7 раза. Система выполняет требование $P95 < 200$ мс при нагрузке до 50 одновременных запросов; при 100 подключениях медиана укладывается в требование, $P95$ превышает его — для таких нагрузок рекомендуется горизонтальное масштабирование API-уровня.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. NBA Stats [Электронный ресурс]. — Официальная статистика Национальной баскетбольной ассоциации. — Режим доступа: <https://www.nba.com/stats> (дата обращения: 29.03.2026).
2. Kubatko, J. et al. A Starting Point for Analyzing Basketball Statistics // Journal of Quantitative Analysis in Sports. — 2007. — Vol. 3, № 3. — Article 1.
3. Basketball-Reference.com [Электронный ресурс]. — Sports Reference LLC. Справочник статистики NBA и исторические таблицы. — Режим доступа: <https://www.basketball-reference.com/> (дата обращения: 29.03.2026).
4. ESPN NBA [Электронный ресурс]. — Новости и обзоры НБА. — Режим доступа: <https://www.espn.com/nba/> (дата обращения: 29.03.2026).
5. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг ; пер. с англ. — 3-е изд. — М. : Вильямс, 2003. — 1440 с.
6. Player Efficiency Rating [Электронный ресурс] // Basketball-Reference.com. — Описание показателя PER. — Режим доступа: <https://www.basketball-reference.com/about/per.html> (дата обращения: 29.03.2026).
7. Myers, D. About Box Plus/Minus (BPM) 2.0 [Электронный ресурс] // Basketball-Reference.com. — Режим доступа: <https://www.basketball-reference.com/about/bpm2.html> (дата обращения: 29.03.2026).
8. Google Developers. Improve Server Response Time [Электронный ресурс]. — Официальная документация Google PageSpeed Insights. — Режим доступа: <https://developers.google.com/speed/docs/insights/Server> (дата обращения: 29.03.2026).
9. Glossary — NBA.com : Stats [Электронный ресурс]. — Термины и определения показателей. — Режим доступа: <https://www.nba.com/stats/help/glossary> (дата обращения: 29.03.2026).
10. Oliver, D. Basketball on Paper: Rules and Tools for Performance Analysis. — Washington, D.C.: Potomac Books, 2004. — 378 p.
11. Hollinger, J. Pro Basketball Forecast. — Washington, D.C.: Potomac Books, 2005. — 432 p.

12. Chen, P. P. The Entity-Relationship Model — Toward a Unified View of Data // ACM Transactions on Database Systems. — 1976. — Vol. 1, № 1. — P. 9–36.
13. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — СПб.: Питер, 2018. — 640 с.
14. Садаладж, П. Дж. NoSQL: новая методология разработки нереляционных баз данных / П. Дж. Садаладж, М. Фаулер. — М.: Вильямс, 2013. — 192 с.
15. ClickHouse Documentation [Электронный ресурс]. — Официальная документация СУБД ClickHouse. — Режим доступа: <https://clickhouse.com/docs/> (дата обращения: 29.03.2026).
16. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт ; пер. с англ. — 8-е изд. — М. : Вильямс, 2006. — 1328 с.
17. Redis Documentation [Электронный ресурс]. — Официальная документация in-мемори хранения Redis. — Режим доступа: <https://redis.io/docs/> (дата обращения: 29.03.2026).
18. PostgreSQL 15 Documentation [Электронный ресурс]. — Официальная документация СУБД PostgreSQL. — Режим доступа: <https://www.postgresql.org/docs/15/> (дата обращения: 29.03.2026).
19. FastAPI Documentation [Электронный ресурс]. — Официальная документация фреймворка FastAPI. — Режим доступа: <https://fastapi.tiangolo.com/> (дата обращения: 29.03.2026).
20. asyncpg Documentation [Электронный ресурс]. — Официальная документация асинхронного драйвера PostgreSQL для Python. — Режим доступа: <https://magicstack.github.io/asyncpg/current/> (дата обращения: 29.03.2026).
21. React Documentation [Электронный ресурс]. — Официальная документация библиотеки React. — Режим доступа: <https://react.dev/> (дата обращения: 29.03.2026).
22. Recharts Documentation [Электронный ресурс]. — Официальная документация библиотеки визуализации данных Recharts. — Режим доступа: <https://recharts.org/> (дата обращения: 29.03.2026).
23. nba_api: an API Client Package to Access the APIs of NBA.com [Электронный ресурс]. — Документация Python-библиотеки для доступа к stats.nba.com. — Режим доступа: https://github.com/swar/nba_api (дата обращения: 29.03.2026).

ПРИЛОЖЕНИЯ

Приложение А. Скрипты создания структуры базы данных

```
CREATE TABLE positions (  
    position_id SERIAL PRIMARY KEY,  
    code VARCHAR(2) UNIQUE NOT NULL,  
    name VARCHAR(20) NOT NULL  
);  
  
CREATE TABLE seasons (  
    season_id SERIAL PRIMARY KEY,  
    label VARCHAR(9) UNIQUE NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE NOT NULL,  
    season_type VARCHAR(15) NOT NULL  
);  
  
CREATE TABLE teams (  
    team_id SERIAL PRIMARY KEY,  
    nba_team_id INT UNIQUE NOT NULL,  
    name VARCHAR(60) UNIQUE NOT NULL,  
    abbreviation CHAR(3) UNIQUE NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    conference VARCHAR(4) NOT NULL,  
    arena_name VARCHAR(80),  
    founded_year SMALLINT,  
    is_active BOOLEAN DEFAULT TRUE  
);
```

Приложение Б. Скрипты создания функций и процедур

```
CREATE OR REPLACE FUNCTION calculate_efg(  
    p_player_id INT,  
    p_season_id INT,  
    p_team_id INT DEFAULT NULL  
)  
RETURNS DECIMAL(5,4)  
LANGUAGE plpgsql  
STABLE  
AS $$
```

```

DECLARE
    v_fgm NUMERIC := 0;
    v_fg3m NUMERIC := 0;
    v_fga NUMERIC := 0;
BEGIN
    SELECT
        COALESCE(SUM(gps.fgm), 0),
        COALESCE(SUM(gps.fg3m), 0),
        COALESCE(SUM(gps.fga), 0)
    INTO v_fgm, v_fg3m, v_fga
    FROM game_player_stats gps
    JOIN games g ON g.game_id = gps.game_id
    WHERE gps.player_id = p_player_id
        AND g.season_id = p_season_id
        AND (p_team_id IS NULL OR gps.team_id = p_team_id);

    IF v_fga = 0 THEN
        RETURN NULL;
    END IF;

    RETURN ROUND((v_fgm + 0.5 * v_fg3m) / v_fga, 4);
END;
$$;

```

Приложение В. Исходный код механизма кэширования

В приложении приведён обобщённый декоратор с ключом по пути и строке запроса. В реализованных маршрутах API (листинг 3.7, технологический раздел) применяется явный формат ключа `leaders:{metric}:{season_id}...`, что обеспечивает точечную инвалидацию по шаблонам при вызове `POST /admin/refresh`.

```

import json
from functools import wraps
from fastapi import Request, Response

def cache_response(ttl_seconds: int = 300):
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            request: Request = kwargs.get("request")
            redis_client = request.app.state.redis

```

```

        cache_key = f"api_cache:{request.url.path}?{request.url
.query}"

        cached_data = await redis_client.get(cache_key)
        if cached_data:
            return Response(
                content=cached_data,
                media_type="application/json"
            )

        response = await func(*args, **kwargs)

        await redis_client.setex(
            cache_key,
            ttl_seconds,
            json.dumps(response)
        )
        return response
    return wrapper
return decorator

```